A Comprehensive Survey on Application Security in Modern Software Development: Taxonomy, Testing Methodologies, and Emerging Challenges

Banan almaqrami

July 24, 2026

Abstract

With the rapid evolution of Software Development Life Cycle (SDLC) models and the adoption of fast-paced deployment frameworks like Agile and DevOps, software delivery speed has become a primary priority. However, this acceleration has frequently led to security being sidelined, resulting in a sharp increase in cyber vulnerabilities at the application layer. Traditionally, security testing was performed during the late stages of development, an approach proven ineffective due to high remediation costs. Consequently, the paradigm has shifted toward **DevSecOps** and **"Shift-Left" security**, mandating the early integration of automated security testing tools. This survey provides a comprehensive review of modern application security (AppSec) testing methodologies, introduces a novel taxonomy classifying existing tools (SAST, DAST, IAST, SCA), analyzes their integration within CI/CD pipelines, and highlights open challenges and future research directions involving artificial intelligence.

1 Introduction

The transition from monolithic architectures to microservices and cloud-native environments has drastically transformed software engineering. While modern development practices enhance scalability and deployment speed, they expand the attack surface. Application layer vulnerabilities remain a primary vector for cyberattacks, making robust security integration imperative.

Historically, security was treated as a gatekeeping phase at the end of the SDLC. This reactive approach caused severe delays and high costs when vulnerabilities were discovered post-deployment. The introduction of **Shift-Left Security** emphasizes embedding security practices into the earliest phases of development, including requirements engineering and coding.

Main Contributions of this Survey:

1. **Taxonomy Development:** Proposes a structured taxonomy categorizing modern AppSec testing methodologies and tools.
2. **Comparative Evaluation:** Evaluates and compares SAST, DAST, IAST, and SCA based on their operational phase, accuracy, and limitations.
3. **Integration Analysis:** Examines how security automation fits within modern CI/CD pipelines under the DevSecOps umbrella.
4. **Future Directions:** Identifies open research challenges, specifically addressing the impact of Generative AI and Large Language Models (LLMs) on application security.

2 Research Methodology

To ensure academic rigor, this survey adopts a systematic literature review approach based on the **PRISMA** guidelines:

- **Digital Libraries:** Queries were executed across IEEE Xplore, ACM Digital Library, SpringerLink, and ScienceDirect.
- **Search Strings:** Boolean operators targeting keywords such as `(Äpplication SecurityÖR ÄppSec)̈ AND (S̈ASTÖR D̈ASTÖR D̈evSecOps)̈ AND (S̈oftware Vulnerabilities)̈`.
- **Inclusion/Exclusion Criteria:** Papers published between 2020 and 2026, peer-reviewed, written in English, and focusing directly on software application security and automated testing mechanisms were included.

3 Taxonomy of Application Security Tools and Techniques

Application security testing tools are classified into four primary categories based on how and when they analyze the software:

3.1 Static Application Security Testing (SAST)

- **Definition:** Often referred to as "white-box testing," SAST analyzes source code, bytecode, or binary code for security vulnerabilities without executing the program.
- **Characteristics:** Operates early in the SDLC (coding phase); detects flaws in structural logic.
- **Limitation:** High rate of **False Positives** and lacks context regarding the runtime environment.

3.2 Dynamic Application Security Testing (DAST)

- **Definition:** Known as "black-box testing," DAST evaluates a running application from the outside by simulating external attacks (e.g., SQL injection, XSS).
- **Characteristics:** Operates during the testing/staging phase; requires no access to source code.
- **Limitation:** Cannot pinpoint the exact line of code causing the vulnerability and covers only executed execution paths.

3.3 Interactive Application Security Testing (IAST)

- **Definition:** A hybrid approach combining elements of SAST and DAST. IAST deploys agents inside the application runtime environment to monitor operations and data flow.
- **Characteristics:** High accuracy, low false-positive rate, provides real-time feedback during functional testing.
- **Limitation:** Requires runtime instrumentation, which may introduce performance overhead.

3.4 Software Composition Analysis (SCA)

- **Definition:** Focuses on identifying and managing third-party open-source components and libraries integrated into the software supply chain.
- **Characteristics:** Essential for preventing supply chain attacks by tracking known vulnerabilities (CVEs) in dependency trees.
- **Limitation:** Relies heavily on up-to-date vulnerability databases.

4 Integration in DevSecOps and Automated Remediation

Integrating AppSec into modern delivery pipelines requires moving away from manual gates toward continuous automated testing.

- **CI/CD Pipeline Integration:** Automated triggers launch SAST upon code commit, SCA during dependency management updates, and DAST/IAST during staging deployments.
- **Auto-Remediation:** Recent trends leverage machine learning models and LLMs to automatically suggest or apply code patches for identified vulnerabilities, significantly reducing Mean Time to Remediate (MTTR).

5 Comparative Analysis of AppSec Methodologies

Table 1: Comparison of Application Security Testing Methodologies

Methodology	Phase in SDLC	Access Required	False Positives	Primary Target
SAST	Coding / Build	Source Code / Binaries	High	Logic flaws, syntax vulnerabilities
DAST	Testing / Staging	Running App (Black-box)	Medium	Runtime vulnerabilities, OWASP Top 10
IAST	Testing / QA	App Runtime + Agents	Low	Deep application logic and data flows
SCA	Build / Dependency	Dependency Manifests	Low	Third-party libraries, supply chain risks

6 Open Challenges and Future Trends

Despite significant advancements, several key challenges remain unresolved in application security research:

1. **False Positives Management:** Balancing high detection rates with low false alarms to prevent developer fatigue.
2. **AI-Generated Code Security:** With developers increasingly using LLMs (like GitHub Copilot) to write code, research indicates that AI-generated code frequently contains security flaws that traditional tools struggle to categorize correctly.
3. **Microservices and Serverless Security:** Securing highly distributed, containerized architectures where traditional perimeter definitions no longer apply.

4. **Real-Time Automated Patching:** Scaling reliable, automated zero-day vulnerability patching without human intervention.

7 Conclusion

Application security has transitioned from an afterthought to a foundational pillar of modern software engineering through the adoption of DevSecOps and automated testing frameworks. This survey systematically categorized AppSec testing techniques into SAST, DAST, IAST, and SCA, highlighting their respective strengths and limitations via comparative analysis. As software complexity grows and generative AI reshapes development workflows, future research must focus on reducing false positives, securing AI-generated code, and optimizing automated remediation across distributed environments.

References

[1] A. AlQahtani, and R. G. Crespo, "A Systematic Literature Review of Automated Vulnerability Detection in Secure Software Development," *IEEE Access*, vol. 11, pp. 45210-45225, 2023.

[2] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley Professional, 2021.

[3] G. McGraw, "Software Security: Building Security In 15 Years Later," *IEEE Security & Privacy*, vol. 18, no. 6, pp. 12-19, 2020.

[4] OWASP Foundation, *OWASP Top 10: The Ten Most Critical Web Application Security Risks*, 2025. [Online]. Available: `https://owasp.org/www-project-top-ten/`

[5] A. Rahman, et al., "Security Analysis of AI-Generated Code: An Empirical Study on GitHub Copilot," in *Proceedings of the ACM/IEEE International Conference on Software Engineering (ICSE)*, 2022, pp. 112-124.

[6] R. Sen, and U. Borooah, "Integrating SAST, DAST, and IAST in CI/CD Pipelines: Challenges and Best Practices," *Journal of Systems and Software*, vol. 208, p. 111850, 2024.

[7] J. Zhao, et al., "A Comprehensive Review of Software Supply Chain Security and Composition Analysis (SCA)," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1890-1908, 2023.